

EXPERIMENT NO. 7

Aim

To study the structure of Solidity programs, basic Solidity constructs, and the compilation and deployment environment, and to gain hands-on experience in creating and interacting with smart contracts using Remix IDE.

Theory

Solidity is a statically typed, high-level programming language primarily used for creating smart contracts on Ethereum and other EVM-compatible blockchain networks. Smart contracts are blockchain-based programs that automatically execute predefined operations when their functions are called and the required conditions are satisfied.

A typical Solidity contract may include a license identifier, compiler version declaration, contract definition, state variables, constructors, modifiers, and functions.

1. Solidity Data Types and Variables

Solidity provides different types of variables for storing and processing information:

- `uint` and `int`: Used to represent unsigned and signed integer values.
- `bool`: Used to store logical values such as `true` or `false`.
- `address`: Represents an Ethereum account or contract address.
- `string`: Used to store text.
- `bytes`: Used for storing raw or binary data.

Variables are broadly categorized according to their lifetime and storage:

- **State variables**: Stored permanently as part of the contract's blockchain state.
- **Local variables**: Created temporarily while a function is being executed.
- **Global variables**: Special variables supplied by the Solidity/EVM environment, such as `msg.sender`, `msg.value`, and `block.timestamp`.

2. Functions in Solidity

Functions define the operations that a smart contract can perform. They can either access the contract's stored information, modify it, or simply perform calculations.

A pure function does not access or change the contract's state.

```
function add(uint a, uint b)
```

```
public
pure
returns(uint)
{
    return a + b;
}
```

A view function can access state variables but cannot change the blockchain state.

```
function getValue()
    public
    view
    returns(uint)
{
    return value;
}
```

3. Function Parameters and Return Values

Functions may receive data through parameters and can send results back using return values. For example, a multiplication function can take two integers and return their product.

```
function multiply(uint a, uint b)
    public
    pure
    returns(uint)
{
    return a * b;
}
```

In this example, **a** and **b** are input parameters, while the calculated product is returned as the output.

4. Function Visibility

Visibility determines where and how a function or variable can be accessed. Solidity supports four main visibility levels:

- **public**: Accessible both from inside the contract and through external calls.
- **private**: Accessible only within the contract where it is declared.
- **internal**: Accessible within the contract and contracts derived from it.
- **external**: Primarily intended to be called from outside the contract.

5. Modifiers

Modifiers are reusable components that can be attached to functions to enforce conditions before or during their execution. They are frequently used to implement authorization and access-control mechanisms.

For example:

```
modifier onlyOwner() {  
    require(msg.sender == owner, "Not owner");  
    _;  
}
```

When this modifier is applied to a function, only the account stored as **owner** is permitted to execute that function.

6. Constructors

A constructor is a special function that runs only once, at the time a contract is deployed. It is commonly used to assign initial values to state variables or establish the contract owner.

```
constructor() {  
    owner = msg.sender;  
}
```

Here, the address that deploys the contract becomes the initial owner.

7. Conditional and Looping Statements

Solidity provides common programming constructs for controlling the flow of execution. These include:

- **if** and **else**
- **for**
- **while**
- **do-while**

Conditional statements allow decisions to be made based on specific conditions, while loops allow a set of instructions to be repeated. Since blockchain computation requires gas, excessive or unbounded loops should be avoided because they can increase transaction costs or cause execution problems.

8. Collections and User-Defined Data Structures

Solidity provides multiple ways to organize data.

Arrays are used to maintain a collection of elements:

```
uint[] public numbers;
```

Mappings associate a key with a corresponding value:

```
mapping(address => uint) public balances;
```

Structs allow different pieces of related information to be combined into a single custom data type:

```
struct Player {  
    string name;  
    uint score;  
}
```

Enums represent a predefined collection of possible states:

```
enum Status {  
    Pending,  
    Active,  
    Closed  
}
```

9. Solidity Data Locations

For reference-type variables, Solidity provides different locations that determine how and where data is stored.

- **storage:** Holds persistent data associated with the contract and remains available between transactions.
- **memory:** Used for temporary data during function execution.
- **calldata:** Contains temporary, read-only data and is commonly used for parameters of external functions.

Choosing an appropriate data location can help improve contract efficiency and reduce unnecessary gas usage.

10. Ether, Wei and Contract Transactions

Ether (ETH) is the native cryptocurrency used within the Ethereum ecosystem. Its smallest denomination is Wei.

1 ETH = 10^{18} Wei

Blockchain transactions are used for activities such as sending Ether, calling smart-contract functions, and modifying contract data. A Solidity function must be marked as **payable** when it is designed to accept Ether.

11. Gas and Transaction Cost

Gas is a measurement of the computational resources required to execute operations on the Ethereum Virtual Machine. Different instructions consume different amounts of gas.

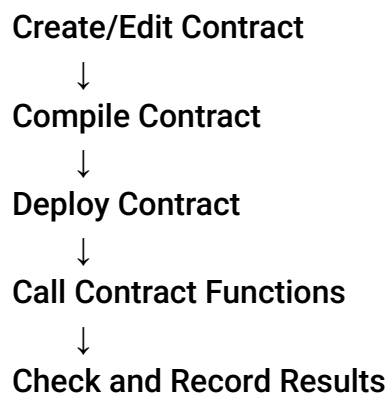
Transaction Fee = Gas Used × Gas Price

Because users pay for computation, smart contracts should be designed efficiently. Operations that require more computation or blockchain storage generally result in higher gas consumption.

12. Remix IDE

Remix IDE is a web-based development platform that provides tools for writing, compiling, deploying, and testing Solidity smart contracts. It allows developers to experiment with contracts without requiring a complete local blockchain setup.

The general workflow followed while working with a Solidity contract is:



For this experiment, the Remix VM is used as the blockchain environment for testing and deployment. The Solidity Basics Course available through Remix LearnETH is used to study Solidity concepts and complete the 19 assigned tutorials. Overall, these concepts form the foundation for writing, deploying, and interacting with Ethereum smart contracts.

OUTPUT : -

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x583...eddC4), 99.999 ETH

Select a function to interact with...

Low level interaction

name: o.string: Alice

Call

Balance: 0 ETH

basicSyntax.sol x basicSyntax_answer.sol primitiveDataTypes.sol basicSyntax_test

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2
3 pragma solidity >=0.8.3 <0.9.0;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x583...eddC4 to: MyContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x69e...858a2 Debug

[call] from: 0x58380da701c568545dcfc803fc8875f56beddC4 to: MyContract.name() data: 0x06f...dde03 Debug

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x583...eddC4), 99.999 ETH

Low level interaction

addr: 0x00

Call

Balance: 0 ETH

basicSyntax.sol basicSyntax_answer.sol primitiveDataTypes.sol basicSyntax_test

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2
3 pragma solidity >=0.8.3 <0.9.0;
4
5 contract PrimitiveDataTypes {
6     address public addr = 0x0000000000000000000000000000000000000000000000000000000000000000;
7     address public newAddr = 0xb000000000000000000000000000000000000000000000000000000000000001;
8
9     int public neg = -1;
10
11     uint8 public newU = 0;
12 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x583...eddC4 to: PrimitiveDataTypes.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x52a...7d39a Debug

[call] from: 0x58380da701c568545dcfc803fc8875f56beddC4 to: PrimitiveDataTypes.addr() data: 0x767...380de Debug

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x583...eddC4), 99.999 ETH

Search functions...

doSomething

blockNumber

x

Gas limit: auto 0

Transact

Balance: 0 ETH

variables.sol x readAndWrite.sol viewAndPure.sol modifiersAndConstructors.sol inputsAndOutputs

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2
3 pragma solidity >=0.8.3 <0.9.0;
4
5 contract Variables {
6     uint public x;
7     uint public blockNumber;
8
9     function doSomething() public {
10         blockNumber = block.number;
11     }
12 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x583...eddC4 to: Variables.doSomething() 0x0fc...5a836 value: 0 wei data: 0x826...32679 logs: 0 hash: 0xb04...2400e Debug

[vm] from: 0x583...eddC4 to: Variables.doSomething() 0x0fc...5a836 value: 0 wei data: 0x826...32679 logs: 0 hash: 0x5af...ca069 Debug

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Ramix VM Osaka

varlet 99.999 ETH

Low level interaction

Value 0 wei

Gas limit auto 0

Transact

Create a DApp with your contracts Start new

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     // You need to send a transaction to write to a state variable.
9     function set(uint _num) public {
10         num = _num;
11     }
12
13     // You can read from a state variable without sending a transaction.
14     function get() public view returns (uint) {
15         return num;
16     }
17 }
```

Explain contract

Listen on all transactions

Filter with transaction hash or address

[vm] From: 0x58...edc4 to: Variables.deploying() 0x58...5463b value: 0 wei data: 0x020...02079 logs: 0 hash: 0x58...c3503

[vm] From: 0x58...edc4 to: ReadAndWrite.constructor() value: 0 wei data: 0x58...28033 logs: 0 hash: 0x58...171f9

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Solidity compiler 0.8.3 Osaka

varlet 99.999 ETH

SimpleStorage 0x4F...22ba

Functions

Select a function to interact with...

Low level interaction

num

uint256: 50

Create a DApp with your contracts Start new

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // num costs 3 gas - this line costs 36 gas - 378000 gas left
6     // State variable to store a number
7     uint public num;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public {
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) {
16        return num;
17    }
18 }
```

Explain contract

Listen on all transactions

Filter with transaction hash or address

[call] From: 0x5802ba78c548549dCf1803Fc387556edc4 to: SimpleStorage.set() data: 0x07...010c

[call] From: 0x5802ba78c548549dCf1803Fc387556edc4 to: SimpleStorage.get() data: 0x07...010c

